Section 8.2: Clean Architecture & Hexagonal Architecture

> ***Module 8 of 8 · Architectural Patterns for Testability and Flexibility***

Executive Overview

Clean and hexagonal architectures solve the same core problem: keeping business logic independent of infrastructure. This independence enables unit testing without databases, swapping third-party providers without touching core logic, and evolving the system without cascading changes. Mastery of ports/adapters, dependency inversion, and Architecture Decision Records (ADRs) is essential for senior-level design interviews.

Part 1: Hexagonal Architecture (Ports & Adapters)

1.1 Core Concept — Dependency Inversion

Traditional layered architecture points dependencies toward the database. Hexagonal architecture reverses this: the application core depends on *abstractions* (ports), and concrete infrastructure implements those abstractions (adapters).

```
Traditional (bad direction):
  HTTP Handler → Application Logic → MySQL Driver → MySQL

Hexagonal (good direction):
  HTTP Adapter → [Inbound Port] → Application Core → [Outbound Port] → DB Adapter
→ MySQL
                                        ↑
                                Business rules live here.
                                No import of MySQL, Stripe, etc.
```

1.2 Port Types

Port Type	Direction	Who Defines It	Who Implements It	Example
Driving (Inbound)	Outside → Core	Core	Adapter (REST, gRPC, CLI)	`PaymentHandler` interface
Driven (Outbound)	Core → Outside	Core	Adapter (Stripe, DB, Email)	`PaymentGateway` interface

> ***Critical rule:*** *The core defines all port interfaces. Infrastructure adapters implement them. The core never imports infrastructure packages.*

1.3 Implementation

```
// ── CORE (no external imports) ──────────────────────────────────

// Driven port — core defines what it needs from the outside
type PaymentGateway interface {
    Charge(ctx context.Context, amount Money, card CardDetails) (ChargeID, error)
    Refund(ctx context.Context, chargeID ChargeID) error
}

type NotificationSender interface {
    Send(ctx context.Context, to EmailAddress, subject, body string) error
}

// Application use case — depends only on abstractions
type PaymentService struct {
    gateway  PaymentGateway       // outbound port
    notifier NotificationSender   // outbound port
    repo     PaymentRepository    // outbound port
}

func (s *PaymentService) ProcessPayment(ctx context.Context, req PaymentRequest)
(Payment, error) {
    chargeID, err := s.gateway.Charge(ctx, req.Amount, req.Card)
    if err != nil {
        return Payment{}, fmt.Errorf("charge failed: %w", err)
    }
    payment := Payment{ID: newPaymentID(), ChargeID: chargeID, Status:
StatusSucceeded}
    if err := s.repo.Save(ctx, payment); err != nil {
        return Payment{}, err
    }
    _ = s.notifier.Send(ctx, req.CustomerEmail, "Payment confirmed",
confirmationBody(payment))
```

```
    return payment, nil
}

// ── ADAPTERS (infrastructure layer) ──────────────────────────────────

// Driven adapter — implements outbound port
type StripeGateway struct {
    client *stripe.Client
}

func (g *StripeGateway) Charge(ctx context.Context, amount Money, card
CardDetails) (ChargeID, error) {
    result, err := g.client.Charges.New(&stripe.ChargeParams{
        Amount:   stripe.Int64(amount.Cents()),
        Currency: stripe.String(amount.Currency),
        Source:   &stripe.SourceParams{Token: stripe.String(card.Token)},
    })
    if err != nil {
        return "", err
    }
    return ChargeID(result.ID), nil
}

// Driving adapter — exposes core to HTTP
type HTTPPaymentHandler struct {
    service *PaymentService   // depends on core, not on Stripe
}

func (h *HTTPPaymentHandler) ServeHTTP(w http.ResponseWriter, r *http.Request) {
    var req PaymentRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, "bad request", http.StatusBadRequest)
        return
    }
    payment, err := h.service.ProcessPayment(r.Context(), req)
    if err != nil {
        http.Error(w, err.Error(), http.StatusInternalServerError)
        return
    }
    json.NewEncoder(w).Encode(payment)
}
```

1.4 Testing Benefits

```
// Test without any real infrastructure
type MockPaymentGateway struct {
    responses map[string]ChargeID
    errors    map[string]error
}

func (m *MockPaymentGateway) Charge(_ context.Context, amount Money, card
CardDetails) (ChargeID, error) {
    if err, ok := m.errors[card.Last4]; ok {
```

```
        return "", err
    }
    return m.responses[card.Last4], nil
}

func (m *MockPaymentGateway) Refund(_ context.Context, id ChargeID) error {
return nil }

func TestPaymentService_SuccessfulCharge(t *testing.T) {
    gateway := &MockPaymentGateway{
        responses: map[string]ChargeID{"4242": "ch_test_123"},
    }
    svc := &PaymentService{gateway: gateway, notifier: &NoopNotifier{}, repo:
&InMemoryRepo{}}

    payment, err := svc.ProcessPayment(context.Background(), PaymentRequest{
        Amount: Money{Amount: 100, Currency: "usd"},
        Card:   CardDetails{Last4: "4242", Token: "tok_visa"},
    })

    assert.NoError(t, err)
    assert.Equal(t, StatusSucceeded, payment.Status)
    // No Stripe API calls made — fast, reliable, free
}
```

Part 2: Clean Architecture / Onion Architecture

2.1 Layer Structure

```
┌──────────────────────────────────────────────────────────────┐
│  Frameworks & Drivers                                        │
│  (HTTP, gRPC, Database drivers, Message brokers, CLI)        │
├──────────────────────────────────────────────────────────────┤
│  Interface Adapters                                          │
│  (Controllers, Presenters, Repository implementations)       │
├──────────────────────────────────────────────────────────────┤
│  Use Cases (Application Business Rules)                      │
│  (Orchestrate entity logic; define port interfaces)          │
├──────────────────────────────────────────────────────────────┤
│  Entities (Enterprise Business Rules)                        │
│  (Domain objects, value objects, domain events)              │
└──────────────────────────────────────────────────────────────┘

Dependency Rule: Source code dependencies point INWARD only.
Inner circles are unaware outer circles exist.
```

2.2 Dependency Rule Violations — Before & After

```
// ❌ WRONG — use case imports database driver
import "database/sql"

func (uc *OrderUseCase) CreateOrder(req CreateOrderRequest) error {
    db, _ := sql.Open("mysql", dsn)
    db.Exec("INSERT INTO orders VALUES (?)", req.ID)  // Violation!
    // Use case now requires a running MySQL to test
}

// ✅ CORRECT — use case defines interface; implementation in outer layer
// Use case layer
type OrderRepository interface {
    Save(ctx context.Context, order Order) error
    FindByID(ctx context.Context, id OrderID) (Order, error)
}

type CreateOrderUseCase struct {
    orders   OrderRepository   // abstract; no MySQL import
    products ProductRepository
}

func (uc *CreateOrderUseCase) Execute(ctx context.Context, req
CreateOrderRequest) (Order, error) {
    product, err := uc.products.FindByID(ctx, req.ProductID)
    if err != nil {
        return Order{}, err
    }
    order := Order{ID: newID(), ProductID: product.ID, Quantity: req.Quantity}
    return order, uc.orders.Save(ctx, order)
}

// Interface adapter layer (outer) — imports MySQL
type MySQLOrderRepository struct{ db *sql.DB }

func (r *MySQLOrderRepository) Save(ctx context.Context, order Order) error {
    _, err := r.db.ExecContext(ctx, "INSERT INTO orders(id, product_id, quantity)
VALUES (?,?,?)",
        order.ID, order.ProductID, order.Quantity)
    return err
}
```

2.3 Hexagonal vs Clean Architecture — Comparison

Aspect	Hexagonal	Clean (Onion)
Mental model	Inside (core) ? Outside (ports & adapters)	Concentric layers; dependencies inward
Terminology	Ports, adapters, driving/driven	

Aspect	Hexagonal	Clean (Onion)
		Entities, use cases, adapters, frameworks
Prescriptiveness	Loose — define your own layers inside	Explicit 4-layer model
Best for	Emphasising testability and provider swap	Emphasising layered separation of concerns
Both achieve	Core isolated from infrastructure	Core isolated from infrastructure

> *In practice, the two are complementary and often combined. Use "hexagonal" vocabulary when discussing provider integration; use "clean" vocabulary when discussing layer separation.*

Part 3: Architecture Decision Records (ADRs)

3.1 Why ADRs Matter

Architecture decisions are made under uncertainty. Without records, teams revisit settled questions, lose institutional knowledge, and make inconsistent choices. ADRs create a *living log* of what was decided, why, and what was rejected.

3.2 Standard ADR Template

```
# ADR-001: Database Selection for User Service

## Status
Accepted  (or: Proposed | Deprecated | Superseded by ADR-007)

## Context
The user service needs durable, consistent storage for account data.
Reads outnumber writes 10:1. We need ACID transactions for account creation.
Team has strong PostgreSQL expertise.

## Decision
Use PostgreSQL 15 over MongoDB 6.

## Rationale
- ACID transactions prevent partial account creation
- Mature JSONB support for flexible profile data
- Existing operational expertise reduces risk
- pg_trgm extension satisfies full-text search needs
```

```
## Consequences
Positive:
- Data integrity guaranteed
- Familiar tooling (pgAdmin, psql, Flyway)
- Rich reporting via SQL

Negative:
- Vertical scaling ceiling (~32 cores practical limit)
- Slower horizontal sharding than MongoDB
- Schema migrations require careful planning

## Alternatives Considered
- MongoDB: Rejected — eventual consistency unacceptable for auth data
- DynamoDB: Rejected — no secondary indexes on arbitrary fields at launch

## Related ADRs
- ADR-002: Caching strategy (Redis) for user session data
- ADR-005: Read replica for analytics queries
```

3.3 ADR Repository Structure

```
/docs/architecture/decisions/
├── 0001-record-architecture-decisions.md     ← meta-ADR
├── 0002-database-postgresql.md
├── 0003-auth-jwt-stateless.md
├── 0004-cache-redis.md
├── 0005-api-gateway-kong.md
└── 0006-event-bus-kafka.md
```

Part 4: Interview Design Problems

Problem 1 — Multi-Provider Payment Service

Question: Apply hexagonal architecture to a payment service that must support Stripe, PayPal, and Braintree. How do you select a provider at runtime? How do you test without calling any provider?

Solution:

```
// Core defines what it needs — one interface for all providers
type PaymentGateway interface {
    Authorize(ctx context.Context, amount Money, card CardDetails)
(AuthorizationID, error)
    Capture(ctx context.Context, authID AuthorizationID) error
    Refund(ctx context.Context, chargeID ChargeID) (RefundID, error)
    HealthCheck(ctx context.Context) error
```

```
}

// Core also defines provider selection logic (abstract)
type GatewaySelector interface {
    Select(ctx context.Context, amount Money, card CardDetails) (PaymentGateway,
error)
}

// Use case — zero knowledge of Stripe/PayPal
type PaymentUseCase struct {
    selector GatewaySelector
    repo     PaymentRepository
}

func (uc *PaymentUseCase) Pay(ctx context.Context, req PaymentRequest) (Payment,
error) {
    gw, err := uc.selector.Select(ctx, req.Amount, req.Card)
    if err != nil {
        return Payment{}, fmt.Errorf("no gateway available: %w", err)
    }
    authID, err := gw.Authorize(ctx, req.Amount, req.Card)
    if err != nil {
        return Payment{}, err
    }
    if err := gw.Capture(ctx, authID); err != nil {
        return Payment{}, err
    }
    payment := Payment{Status: Succeeded, AuthID: authID}
    return payment, uc.repo.Save(ctx, payment)
}

// Infrastructure: smart selector based on cost + health
type CostBasedSelector struct {
    gateways map[ProviderName]PaymentGateway
    costs    map[ProviderName]float64  // fee percentage
}

func (s *CostBasedSelector) Select(ctx context.Context, amount Money, _
CardDetails) (PaymentGateway, error) {
    var best ProviderName
    var bestCost float64 = math.MaxFloat64

    for name, gw := range s.gateways {
        if err := gw.HealthCheck(ctx); err != nil {
            continue  // skip unhealthy providers
        }
        if s.costs[name] < bestCost {
            bestCost = s.costs[name]
            best = name
        }
    }
    if best == "" {
        return nil, errors.New("all gateways unhealthy")
    }
    return s.gateways[best], nil
```

```
}

// Adding a new provider = implement one interface, wire in selector
type BraintreeGateway struct{ client *braintree.Client }
// ... implements PaymentGateway — core code never changes
```

Test strategy: Inject `MockGateway` implementing `PaymentGateway`. Test routing logic with `MockSelector`. Zero network calls in unit tests.

Problem 2 — Replacing a Database Without Touching Business Logic

Question: You need to migrate from MySQL to DynamoDB for the Order service. How does clean architecture make this safe?

Solution:

```
// Step 1: Existing interface in use-case layer (already abstracted)
type OrderRepository interface {
    Save(ctx context.Context, order Order) error
    FindByID(ctx context.Context, id OrderID) (Order, error)
    FindByCustomer(ctx context.Context, customerID CustomerID) ([]Order, error)
}

// Step 2: New DynamoDB implementation — use case layer never changes
type DynamoOrderRepository struct {
    client    *dynamodb.Client
    tableName string
}

func (r *DynamoOrderRepository) Save(ctx context.Context, order Order) error {
    item, err := attributevalue.MarshalMap(order)
    if err != nil {
        return err
    }
    _, err = r.client.PutItem(ctx, &dynamodb.PutItemInput{
        TableName: &r.tableName,
        Item:      item,
    })
    return err
}

// Step 3: Migration strategy — dual-write adapter
type MigrationOrderRepository struct {
    mysql   OrderRepository  // old
    dynamo  OrderRepository  // new
    readNew bool             // feature flag: when true, read from DynamoDB
}

func (r *MigrationOrderRepository) Save(ctx context.Context, order Order) error {
    if err := r.mysql.Save(ctx, order); err != nil {
```

```
        return err  // fail on primary
    }
    _ = r.dynamo.Save(ctx, order)  // best-effort to new store
    return nil
}

func (r *MigrationOrderRepository) FindByID(ctx context.Context, id OrderID)
(Order, error) {
    if r.readNew {
        return r.dynamo.FindByID(ctx, id)
    }
    return r.mysql.FindByID(ctx, id)
}
```

Flip `readNew` via a feature flag. Roll back by switching the flag. Remove MySQL adapter when migration is complete. **The use case layer has zero changes across the entire migration.**

Problem 3 — Designing an ADR for a Contentious Decision

Question: Your team is split between REST and gRPC for an internal service API. Walk through how you'd document this as an ADR and what the decision criteria should be.

Solution:

```
# ADR-012: Internal Service Communication Protocol

## Status
Accepted

## Context
Service A (order processing) must call Service B (inventory) synchronously.
Internal traffic only; no browser clients.
Latency budget: < 20ms p99.
Team A uses Go; Team B uses Java.
Both services will be called by a third Go service in Q3.

## Decision
Use gRPC with Protocol Buffers over REST/JSON.

## Rationale
Performance measurements (our benchmark, 10K req/s):
  REST/JSON:  avg 4.2ms, p99 18ms, serialization: 1.8ms
  gRPC/Proto: avg 1.1ms, p99  6ms, serialization: 0.2ms
gRPC serialization is ~9x faster; fits latency budget with headroom.

Additional factors:
  - Protobuf schema acts as enforced contract (no runtime surprises)
  - Bidirectional streaming available for future bulk reservation calls
  - Go and Java both have mature gRPC libraries
```

```
## Consequences
Positive:
  - Meets latency requirement
  - Strong typing catches contract breaks at compile time
  - Generated client code reduces boilerplate

Negative:
  - Binary protocol harder to debug (use grpcurl / grpc-gateway for dev)
  - Browser clients cannot call gRPC directly (mitigation: grpc-gateway if needed)
  - Team must learn Protobuf schema evolution rules (field numbers, deprecation)

## Alternatives Considered
  - REST/JSON: Simpler tooling, but p99 latency borderline at 18ms; no schema
enforcement
  - GraphQL: Overkill for internal calls; no performance advantage over REST
  - Kafka async: Rejected — this path requires synchronous response

## Review Trigger
Revisit if a browser client needs to call Service B directly.
```

Quick Reference

Pattern	Problem Solved	Key Mechanism
Hexagonal (Ports & Adapters)	Core depends on infrastructure	Define port interfaces in core; adapters implement them
Clean Architecture	Layer violations; untestable code	Dependency rule: inward only
ADR	Decision loss; repeated arguments	Document context, decision, consequences, alternatives
Mock Adapter	Test without infrastructure	Implement port interface with in-memory behaviour
Migration Adapter	Safe infrastructure swap	Dual-write during transition; toggle reads via flag